

8-4 · Streamlit 기반의 챗봇 인터페이스 개발 · 정리·회고 0.5H

정리·회고

챗봇 구조를 한 표로, 8-5 최종일 시나리오 예고

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

30분 뒤에는

- ① 챗봇 뼈대를 「하려는 일 ↔ 코드」 한 표로 정리한다
- ② 오늘의 체크리스트로 빠진 단계를 스스로 찾는다
- ③ 자주 겪는 오류 네 가지의 원인과 해결을 안다
- ④ 8-5 최종일에 필요한 것이 켜져 있는지 확인한다

오늘 파일을 지우지 마세요 — 8-5의 출발점입니다

하려는 일 ↔ 코드 — 여덟 줄로 정리

하려는 일	코드
이력 저장소 만들기	if "messages" not in st.session_state: ...
이력 다시 그리기	for m in st.session_state.messages: ...
입력 받기	st.chat_input("...")
말풍선	with st.chat_message("user"): st.markdown(...)
연결 캐시	@st.cache_resource
값 캐시	@st.cache_data
즉시 재실행	st.rerun()
접는 영역	with st.expander("근거"): ...

사이드바는 with st.sidebar: ... · 실행은 streamlit run app.py

열세 개를 여덟 줄로 — 스스로 확인해 보세요

- 1 상호작용마다 스크립트가 다시 실행되는 것을 화면으로 확인했다
- 2 일반 변수와 `session_state` 의 차이를 직접 눌러 봤다
- 3 다시 그리는 for 루프를 지워 대화가 사라지는 것을 재현했다
- 4 `chat_input / chat_message` 로 말풍선을 구분해 그렸다
- 5 사이드바·대화 지우기·`@st.cache_resource` 효과를 확인했다
- 6 화면을 붙이기 전에 검색만 따로 시험했다
- 7 임베딩 → `query_points` → LLM 호출 3단계를 코드에서 짚을 수 있다
- 8 근거 표시·개수 비교·약한 근거 경고·모른다고 답하기를 확인했다

증상으로 찾는 원인과 해결 — 네 가지

증상	원인	해결
메시지를 보내면 이전 대화가 사라짐	일반 변수에 담았거나 다시 그리는 루프 없음	이력은 session_state 에 상단 루프로 전체 다시 그림
Qdrant 연결 거부 connection refused	챗봇은 호스트에서 실행 — host="qdrant" 는 못 찾음	localhost:6333 으로 · compose 의 6333 포트 확인
OpenAI 인증 오류 (api_key 관련)	export 안 한 터미널에서 streamlit run 실행	같은 터미널에서 export 뒤 streamlit 재시작
코드를 고쳤는데 화면이 그대로	캐시된 결과가 남았거나 브라우저가 옛 화면	우측 상단 Clear cache 또는 streamlit 재시작

docker compose ps 에서 qdrant 의 포트 매핑과 기동 상태를 함께 본다 · 키 값은 코드에 하드코딩하지 않는다

자주 나오는 질문 네 가지

- 1 여러 사람이 동시에 쓰면 대화가 섞이나요?**
섞이지 않는다 — session_state 는 브라우저 세션 단위. 다만 서버 자원은 함께 쓴다
- 2 대화를 영구 보관하려면?**
DB 에 저장한다 — session_state 는 서버 재시작에 사라진다
- 3 배포는 어떻게 하나요?**
Streamlit Community Cloud, 또는 Docker 이미지로 만들어 서버에 — 이번 과정 범위 밖
- 4 응답이 느린데요?**
임베딩·검색·생성 세 호출이 순차로 — 스트리밍으로 체감 속도를 올리는 것이 가장 효과적

8-5 까지 켜 두고 남겨 둘 것

- 1 오늘 파일을 지우지 않는다**
app.py 가 8-5 최종본의 출발점 — `cd ~/rag-chatbot && ls -la`
- 2 docker ps 로 qdrant 가 Up 인지**
`docker ps --format "table {{.Names}}\t{{.Status}}" | head`
- 3 app.py 가 있고 qdrant 가 Up 이면**
8-5 준비 완료

8-5 할 일

최종본 다듬기
시연·제출

8-5: 하이브리드 검색·출처 표시·설정 화면으로 다듬기 → 전 주차를 잇는 시연 시나리오 → 최종 산출물(DAG + 챗봇 앱 + 문서) 제출

다음 회차 예고

8-5 · 최종 RAG 챗봇 개발 — 파이프라인 통합

01

이론 1H

최종 아키텍처
점검

02

실습 1.5H

DAG + RAG 챗봇
통합 구축

03

실습 1H

엔드투엔드
검증·시연

04

정리 0.5H

과정 회고 ·
산출물 제출 확인

준비: Airflow 전 서비스와 Qdrant(docker compose ps) · lecture_docs 적재분 · 8-4 가상환경과 OPENAI_API_KEY — 내려가 있으면 먼저 올리기

8-4 · 정리·회고 0.5H 마무리

다음 회차는 8-5 최종 RAG 챗봇

8주 전 구간(수집→Lake→변환→VectorDB→챗봇)을 하나의 시스템으로 묶고, 새 문서가 답변에 반영되는 시나리오로 검증합니다

✓ 8주 전 구간을 한 아키텍처 그림으로

✓ 스케줄·재시도·역등 적재를 갖춘 최종 DAG

✓ 새 문서 → DAG 실행 → 챗봇 답변 반영 검증

✓ 코드·README·requirements·env.example 패키징

준비할 것

Airflow·Qdrant 기동 · app.py 그대로